



南开大学  
Nankai University

## 数据结构实验报告六

### 队列与树

学号：2412266

姓名：杨滢

专业：信息安全

学院：密码与网络安全学院

# 目录

|                                   |           |
|-----------------------------------|-----------|
| <b>1 题目一</b>                      | <b>2</b>  |
| 1.1 题目表述                          | 2         |
| 1.2 代码 1 的测试用例                    | 2         |
| 1.3 思路                            | 2         |
| 1.3.1 循环数组实现双端队列                  | 2         |
| 1.3.2 命令读取                        | 2         |
| 1.3.3 IsEmpty 和 IsFull 命令处理       | 3         |
| 1.3.4 Left 和 Right 命令处理           | 3         |
| 1.3.5 AddLeft 和 AddRight 命令处理     | 3         |
| 1.3.6 DeleteLeft 和 DeleteRight 命令 | 3         |
| 1.4 代码                            | 3         |
| 1.5 测试用例截图                        | 7         |
| 1.6 复杂度分析                         | 8         |
| <b>2 题目二</b>                      | <b>8</b>  |
| 2.1 题目表述                          | 8         |
| 2.2 代码 2 的测试用例                    | 8         |
| 2.3 思路                            | 8         |
| 2.3.1 中缀表达式转后缀表达式                 | 8         |
| 2.3.2 构建表达式树                      | 9         |
| 2.3.3 树形输出                        | 9         |
| 2.4 代码                            | 9         |
| 2.5 测试用例截图                        | 12        |
| <b>3 题目三</b>                      | <b>13</b> |
| 3.1 题目表述                          | 13        |
| 3.2 代码 3 的测试用例                    | 13        |
| 3.3 思路                            | 13        |
| 3.3.1 输入解析                        | 13        |
| 3.3.2 子树交换                        | 14        |
| 3.3.3 叶节点处理                       | 14        |
| 3.3.4 树形输出                        | 14        |
| 3.4 代码                            | 14        |
| 3.5 测试用例截图                        | 19        |

# 1 题目一

## 1.1 题目表述

1. 所谓双端队列 (double-ended queue, deque), 就是在列表的两端都可以插入和删除数据。因此它允许的操作有 Create、IsEmpty、IsFull、Left、Right、AddLeft、AddRight、DeleteLeft、DeleteRight。使用循环数组方式实现双端队列, 要求实现上述操作, 并实现一个 Print 输出操作, 能将队列由左至右的次序输出于一行, 元素间用空格间隔。队列元素类型设为整型。

输入: input.txt, 给出一个操作序列, 可能是 Create、Print 之外的任何操作, 需要的情况下, 会给出参数。最后以关键字 “End” 结束。

输出: 程序开始执行时, 队列设置为空, 按输入顺序执行操作, 每个操作执行完后, 将结果输出于一行。对于错误命令, 输出 “WRONG”。对 IsEmpty 和 IsFull 命令, 试情况输出 “Yes” 或 “No”。对 Left 和 Right 命令, 若队列空, 输出 “EMPTY”, 否则输出对应队列元素。对 Add 命令, 若队列满, 输出 “FULL”, 否则调用 Print, 输出队列所有元素。对 Del 命令, 若队列空, 输出 “EMPTY”, 否则输出所有元素。元素间用空格间隔, 最后一个元素后不能有空格。最后输出一个回车。

## 1.2 代码 1 的测试用例

1. AddLeft 5, 输出 → 队列: [5]
2. AddRight 10, 输出 → 队列: [5, 10]
3. AddLeft 3, 输出 → 队列: [3, 5, 10]
4. Left → 输出左端元素 3
5. Right → 输出右端元素 10
6. DeleteLeft, 输出 → 队列: [5, 10]
7. DeleteRight, 输出 → 队列: [5]
8. IsEmpty → 队列非空

## 1.3 思路

### 1.3.1 循环数组实现双端队列

用数组模拟环形结构, 指针到达边界时循环回到另一侧  
指针移动规则:

- 向左移动:  $\text{front} = (\text{front} - 1 + \text{capacity}) \% \text{capacity}$
- 向右移动:  $\text{rear} = (\text{rear} + 1) \% \text{capacity}$

### 1.3.2 命令读取

使用 if-else 语句, 分支语句条件判断定位到具体的操作命令。

### 1.3.3 IsEmpty 和 IsFull 命令处理

- IsEmpty 命令通过检查 size 变量是否为 0 来判断队列是否为空，输出相应的"Yes" 或"No"。
- IsFull 命令通过比较 size 和 capacity 的值来判断队列是否已满：capacity 固定为 3，当 size 等于 3 时队列满。

### 1.3.4 Left 和 Right 命令处理

首先检查队列是否为空。

- Left 命令在队列非空时返回 front 指针指向的元素，队列空时输出"EMPTY"。
- Right 命令在队列非空时返回 rear 指针前一个位置的元素，使用模运算处理循环数组的边界情况，队列空时输出"EMPTY"。

### 1.3.5 AddLeft 和 AddRight 命令处理

首先读取数值参数，然后检查队列是否已满。如果队列已满则输出"FULL" 并拒绝操作；否则执行相应的添加操作。两种添加操作成功后都调用 Print 方法输出当前队列。

- AddLeft 通过将 front 指针向前移动并在新位置存入值来实现左端添加。
- AddRight 通过在 rear 位置存入值并将 rear 指针向后移动来实现右端添加。

### 1.3.6 DeleteLeft 和 DeleteRight 命令

首先检查队列是否为空。如果队列空则输出"EMPTY"；否则执行相应的删除操作。两种添加操作成功后都调用 Print 方法输出当前队列。

- DeleteLeft 通过将 front 指针向后移动来实现左端删除。
- DeleteRight 通过将 rear 指针向前移动来实现右端删除。

## 1.4 代码

```
1 #include <iostream>
2 #include <fstream>
3 #include <string>
4 #include <sstream>
5 using namespace std;
6 class Deque {
7 private:
8     int* data;
9     int capacity;
10    int front;
11    int rear;
12    int size;
```

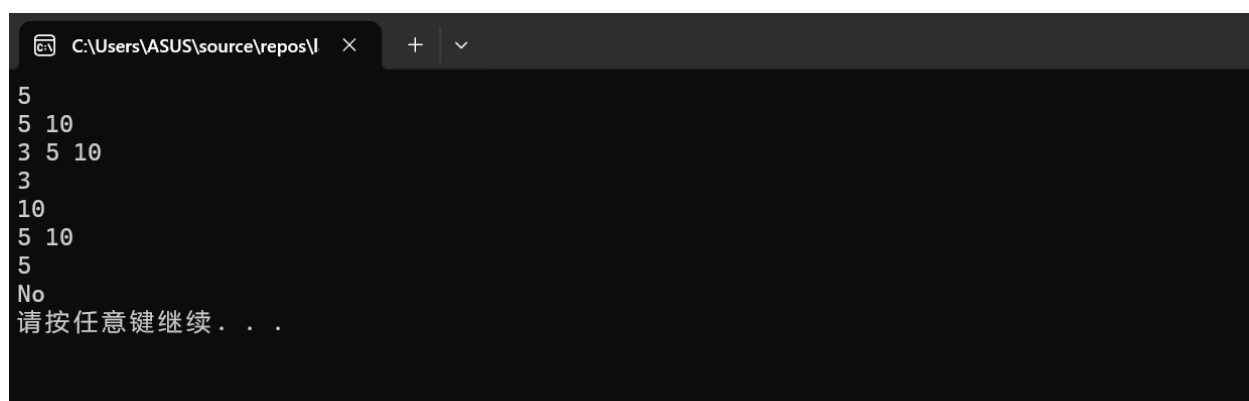
```
13 public:
14     Deque() : data(nullptr), capacity(3), front(0), rear(0), size(0) {
15         data = new int[capacity];
16     }
17     ~Deque() {
18         if (data != nullptr) {
19             delete[] data;
20         }
21     }
22     bool IsEmpty() {
23         return size == 0;
24     }
25     bool IsFull() {
26         return size == capacity;
27     }
28     int Left() {
29         if (IsEmpty()) return -1;
30         return data[front];
31     }
32     int Right() {
33         if (IsEmpty()) return -1;
34         return data[(rear - 1 + capacity) % capacity];
35     }
36     bool AddLeft(int value) {
37         if (IsFull()) return false;
38         front = (front - 1 + capacity) % capacity;
39         data[front] = value;
40         size++;
41         return true;
42     }
43     bool AddRight(int value) {
44         if (IsFull()) return false;
45         data[rear] = value;
46         rear = (rear + 1) % capacity;
47         size++;
48         return true;
49     }
50     bool DeleteLeft() {
51         if (IsEmpty()) return false;
52         front = (front + 1) % capacity;
53         size--;
54         return true;
```

```
55     }
56     bool DeleteRight() {
57         if (IsEmpty()) return false;
58         rear = (rear - 1 + capacity) % capacity;
59         size--;
60         return true;
61     }
62     void Print() {
63         if (IsEmpty()) return;
64         for (int i = 0; i < size; i++) {
65             int index = (front + i) % capacity;
66             cout << data[index];
67             if (i < size - 1) {
68                 cout << " ";
69             }
70         }
71     }
72 };
73 int main() {
74     Deque deque;
75     string line;
76     ifstream infile("input.txt");
77     while (getline(infile ? infile : cin, line)) {
78         if (line == "End")
79             break;
80
81         stringstream ss(line);
82         string command;
83         ss >> command;
84
85         if (command == "IsEmpty") {
86             cout << (deque.IsEmpty() ? "Yes" : "No") << endl;
87         }
88         else if (command == "IsFull") {
89             cout << (deque.IsFull() ? "Yes" : "No") << endl;
90         }
91         else if (command == "Left") {
92             if (deque.IsEmpty()) {
93                 cout << "EMPTY" << endl;
94             }
95             else {
96                 cout << deque.Left() << endl;
```

```
97     }
98 }
99 else if (command == "Right") {
100     if (deque.IsEmpty()) {
101         cout << "EMPTY" << endl;
102     }
103     else {
104         cout << deque.Right() << endl;
105     }
106 }
107 else if (command == "AddLeft") {
108     int value;
109     if (ss >> value) {
110         if (deque.IsFull()) {
111             cout << "FULL" << endl;
112         }
113         else {
114             deque.AddLeft(value);
115             deque.Print();
116             cout << endl;
117         }
118     }
119     else {
120         cout << "WRONG" << endl;
121     }
122 }
123 else if (command == "AddRight") {
124     int value;
125     if (ss >> value) {
126         if (deque.IsFull()) {
127             cout << "FULL" << endl;
128         }
129         else {
130             deque.AddRight(value);
131             deque.Print();
132             cout << endl;
133         }
134     }
135     else {
136         cout << "WRONG" << endl;
137     }
138 }
```

```
139     else if (command == "DeleteLeft") {
140         if (deque.IsEmpty()) {
141             cout << "EMPTY" << endl;
142         }
143         else {
144             deque.DeleteLeft();
145             deque.Print();
146             cout << endl;
147         }
148     }
149     else if (command == "DeleteRight") {
150         if (deque.IsEmpty()) {
151             cout << "EMPTY" << endl;
152         }
153         else {
154             deque.DeleteRight();
155             deque.Print();
156             cout << endl;
157         }
158     }
159     else {
160         cout << "WRONG" << endl;
161     }
162 }
163 system("pause");
164 return 0;
165 }
```

### 1.5 测试用例截图



```
C:\Users\ASUS\source\repos\l  ×  +  v
5
5 10
3 5 10
3
10
5 10
5
No
请按任意键继续...
```



## 1.6 复杂度分析

| 操作                            | 时间复杂度  | 空间复杂度  |
|-------------------------------|--------|--------|
| IsEmpty、IsFull                | $O(1)$ | $O(1)$ |
| Left、Right                    | $O(1)$ | $O(1)$ |
| AddLeft、AddRight + Print      | $O(n)$ | $O(1)$ |
| DeleteLeft、DeleteRight+ Print | $O(n)$ | $O(1)$ |

表 1: 双端队列操作复杂度分析

## 2 题目二

### 2.1 题目表述

输入一个中缀表达式，构造表达式树，以文本方式输出树结构。

### 2.2 代码 2 的测试用例

1. a 输出: a
2. a\*(b+c) 输出构造成由字符和空格表示的树结构如:

```
      c
    +
     b
  *
  a
```

3. (a+b)\*(c-d)/e 输出: 略
4. a+(b-(c+d)\*e) 输出: 略

### 2.3 思路

将中缀表达式转换为表达式树，并以缩进树形格式输出。

#### 2.3.1 中缀表达式转后缀表达式

为便于输出树，首先将中缀表达式转后缀表达式 (infixToPostfix),。转换规则:

1. 利用 salnum() 函数检查字符是否为字母或数字，操作数直接输出
2. 运算符根据优先级入栈出栈
3. 括号特殊处理

### 2.3.2 构建表达式树

1. 从左到右扫描后缀表达式，利用 `salnum()` 函数检查字符是否为字母或数字
2. 遇到操作数：创建节点并入栈
3. 遇到运算符：弹出两个操作数，创建运算符节点，连接为子树
4. 最终栈顶就是完整的表达式树

### 2.3.3 树形输出

由于右子树在上方输出，根节点在中间，左子树在下方输出，并且要通过缩进表示层次关系，我们依据右子树 → 根节点 → 左子树的顺序遍历。

- `isOperator()`: 判断字符是否为运算符
- `getPriority()`: 运算符优先级 ( $> * / > + -$ )
- `infixToPostfix()`: 使用栈实现中缀转后缀
- `buildTreeFromPostfix()`: 使用栈构建表达式树
- `printTree()`: 递归输出树形结构

两次使用栈结构，采取深度优先的树遍历。

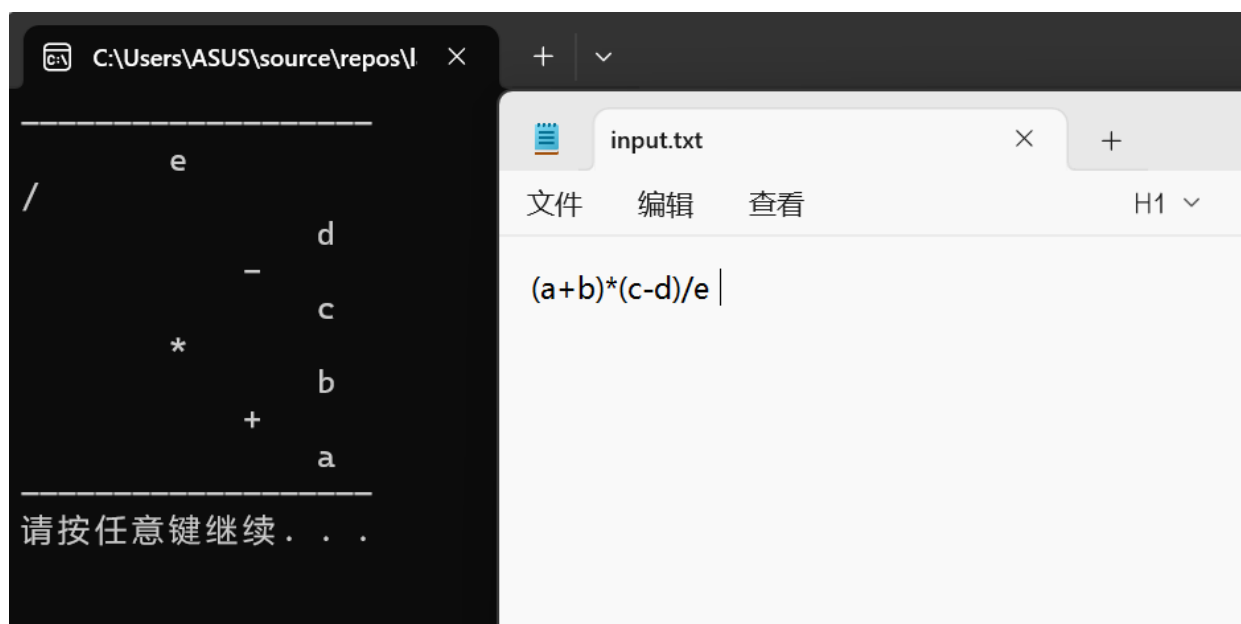
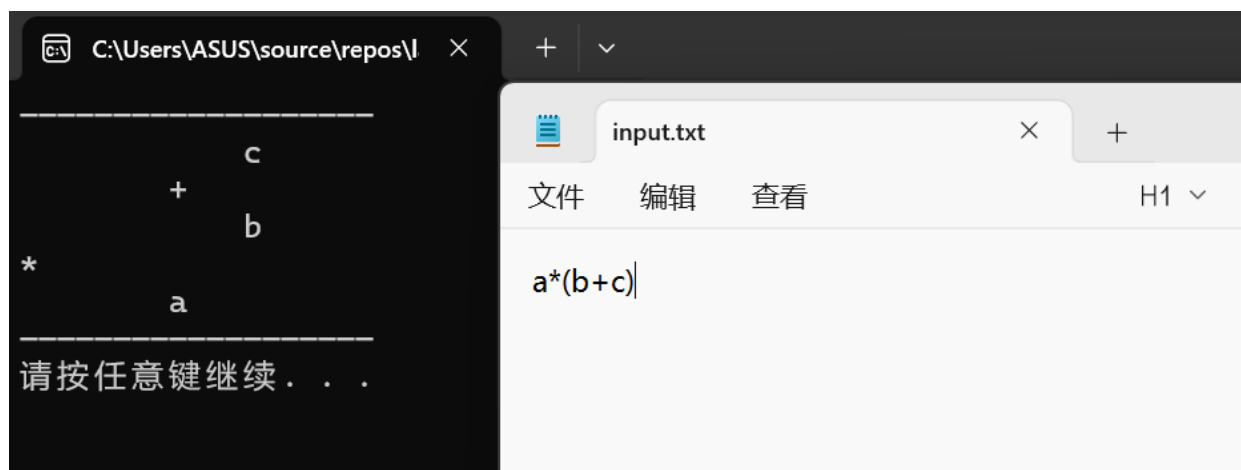
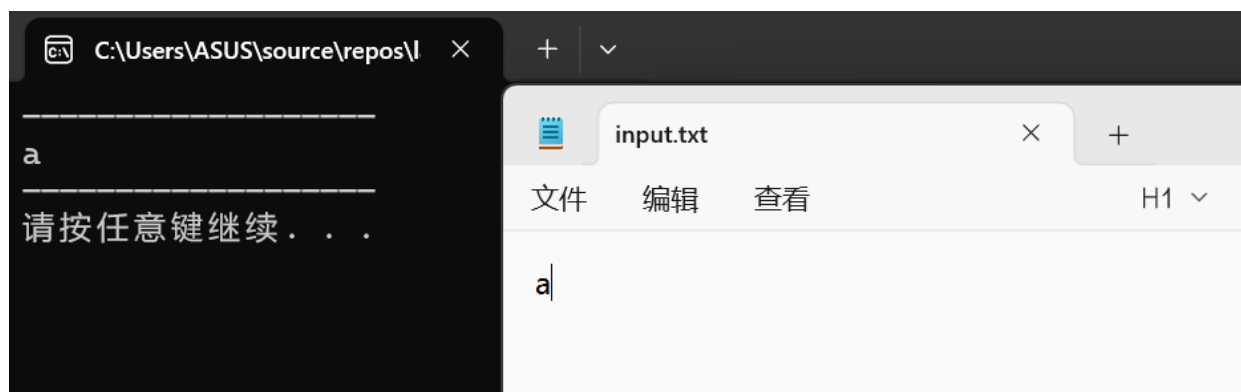
## 2.4 代码

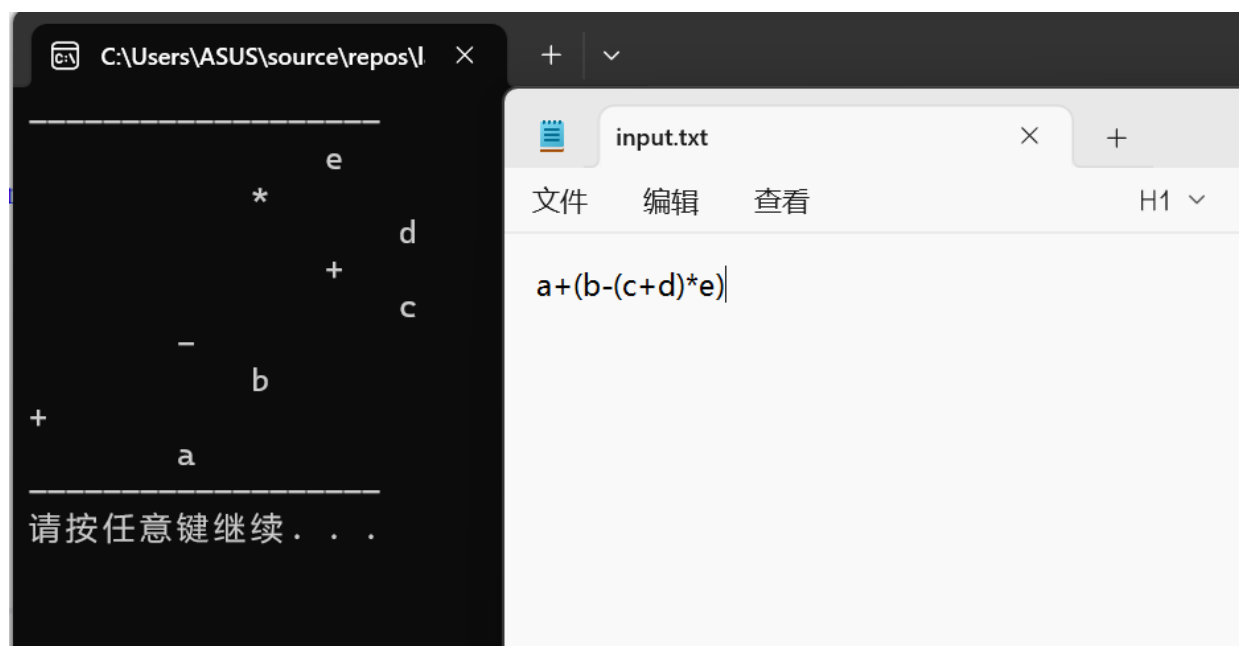
```
1 #include <iostream>
2 #include <fstream>
3 #include <stack>
4 #include <string>
5 #include <cctype>
6 using namespace std;
7 struct Node {
8     char data;
9     Node* left;
10    Node* right;
11    Node(char val) : data(val), left(nullptr), right(nullptr) {}
12 };
13 bool isOperator(char c) {
14     return c == '+' || c == '-' || c == '*' || c == '/' || c == '^';
15 }
16 int getPriority(char op) {
17     if (op == '^') return 3;
18     if (op == '*' || op == '/') return 2;
19     if (op == '+' || op == '-') return 1;
```

```
20     return 0;
21 }
22 string infixToPostfix(const string& infix) {
23     stack<char> st;
24     string postfix = "";
25     for (char c : infix) {
26         if (isalnum(c)) {
27             postfix += c;
28         }
29         else if (c == '(') {
30             st.push(c);
31         }
32         else if (c == ')') {
33             while (!st.empty() && st.top() != '(') {
34                 postfix += st.top();
35                 st.pop();
36             }
37             st.pop();
38         }
39         else if (isOperator(c)) {
40             while (!st.empty() && getPriority(st.top()) >= getPriority(c)) {
41                 postfix += st.top();
42                 st.pop();
43             }
44             st.push(c);
45         }
46     }
47     while (!st.empty()) {
48         postfix += st.top();
49         st.pop();
50     }
51     return postfix;
52 }
53 Node* buildTreeFromPostfix(const string& postfix) {
54     stack<Node*> st;
55     for (char c : postfix) {
56         if (isalnum(c)) {
57             st.push(new Node(c));
58         }
59         else if (isOperator(c)) {
60             Node* node = new Node(c);
61             node->right = st.top(); st.pop();
```

```
62         node->left = st.top(); st.pop();
63         st.push(node);
64     }
65 }
66 return st.top();
67 }
68 void printTree(Node* root, int level = 0, string prefix = "") {
69     if (root == nullptr) return;
70     printTree(root->right, level + 1, " ");
71     cout << string(level * 4, ' ') << prefix << root->data << endl;
72     printTree(root->left, level + 1, " ");
73 }
74
75 int main() {
76     ifstream inputFile("input.txt");
77     string infix;
78     if (!inputFile.is_open()) {
79         cout << "WRONG" << endl;
80         return 1;
81     }
82     getline(inputFile, infix);
83     inputFile.close();
84     string postfix = infixToPostfix(infix);
85     Node* root = buildTreeFromPostfix(postfix);
86     cout << "          " << endl;
87     printTree(root);
88     cout << "          " << endl;
89     system("pause");
90     return 0;
91 }
```

## 2.5 测试用例截图





### 3 题目三

#### 3.1 题目表述

编写二叉树类的成员函数，分别实现以下功能：

1. 统计二叉树的叶节点的数目
2. 交换二叉树中所有节点的左右子树
3. 按层次顺序遍历二叉树：首先访问根节点，然后是它的两个孩子节点，然后是孙子节点，依此类推
4. 求二叉树的宽度，即同一层次上最多的节点数

输入一棵二叉树，程序依次显示上述各项处理的结果。

#### 3.2 代码 3 的测试用例

1.  $A(B(D,E),C(F))$
2.  $A(B(C(D),),))$

#### 3.3 思路

##### 3.3.1 输入解析

这里利用括号解析输入

- 遇到字母：创建新节点
- 遇到 (：递归构建左子树

- 遇到,: 递归构建右子树
- 遇到): 结束当前子树构建

### 3.3.2 子树交换

交换当前节点的左右指针, 利用递归交换。

### 3.3.3 叶节点处理

- 统计数目 (leaves): 递归遍历, 叶节点返回 1, 非叶节点返回左右子树叶节点和
- 收集叶节点 (getLeafNodes): 深度优先遍历, 遇到叶节点时记录其值

### 3.3.4 树形输出

沿用题目二的输出形式, 用缩进来表示树实现交换二叉树中所有节点的左右子树后的输出。

## 3.4 代码

```
1 #include <iostream>
2 #include <fstream>
3 #include <string>
4 #include <vector>
5 #include <queue>
6 using namespace std;
7 struct Node {
8     char val;
9     Node* l;
10    Node* r;
11    Node(char x) : val(x), l(nullptr), r(nullptr) {}
12 };
13 class Tree {
14 private:
15     Node* root;
16     Node* build(string s, int& idx) {
17         if (idx >= s.length() || s[idx] == ',' || s[idx] == ')')
18             return nullptr;
19         char c = s[idx++];
20         Node* node = new Node(c);
21         if (idx < s.length() && s[idx] == '(') {
22             idx++;
23             node->l = build(s, idx);
24             if (idx < s.length() && s[idx] == ',') {
25                 idx++;
```

```
26         node->r = build(s, idx);
27     }
28     if (idx < s.length() && s[idx] == ')') {
29         idx++;
30     }
31 }
32 return node;
33 }
34 int leaves(Node* n) {
35     if (!n) return 0;
36     if (!n->l && !n->r) return 1;
37     return leaves(n->l) + leaves(n->r);
38 }
39 void getLeafNodes(Node* n, vector<char>& result) {
40     if (!n) return;
41     if (!n->l && !n->r) {
42         result.push_back(n->val);
43         return;
44     }
45     getLeafNodes(n->l, result);
46     getLeafNodes(n->r, result);
47 }
48 void swap(Node* n) {
49     if (!n) return;
50     Node* t = n->l;
51     n->l = n->r;
52     n->r = t;
53     swap(n->l);
54     swap(n->r);
55 }
56 void print(Node* n, int lev = 0) {
57     if (!n) return;
58     print(n->r, lev + 1);
59     cout << string(lev * 4, ' ') << n->val << endl;
60     print(n->l, lev + 1);
61 }
62 void del(Node* n) {
63     if (!n) return;
64     del(n->l);
65     del(n->r);
66     delete n;
67 }
```



```
68 public:
69     Tree() : root(nullptr) {}
70     ~Tree() { del(root); }
71     void buildFromString(string s) {
72         int idx = 0;
73         root = build(s, idx);
74     }
75     int countLeaves() {
76         return leaves(root);
77     }
78     vector<char> getLeafNodes() {
79         vector<char> result;
80         getLeafNodes(root, result);
81         return result;
82     }
83     void swapAll() {
84         swap(root);
85     }
86     void printTree() {
87         if (!root) {
88             cout << "树为空" << endl;
89             return;
90         }
91         cout << "          " << endl;
92         print(root);
93         cout << "          " << endl;
94     }
95     void levelOrder() {
96         if (!root) {
97             cout << "树为空" << endl;
98             return;
99         }
100         queue<Node*> q;
101         q.push(root);
102         while (!q.empty()) {
103             Node* cur = q.front(); q.pop();
104             cout << cur->val << " ";
105             if (cur->l) q.push(cur->l);
106             if (cur->r) q.push(cur->r);
107         }
108         cout << endl;
109     }
```

```
110 void printLevelNodes() {
111     if (!root) {
112         cout << "树为空" << endl;
113         return;
114     }
115     queue<Node*> q;
116     q.push(root);
117     int level = 1;
118     while (!q.empty()) {
119         int sz = q.size();
120         cout << "第" << level << "层: " << sz << endl;
121         for (int i = 0; i < sz; i++) {
122             Node* cur = q.front(); q.pop();
123             if (cur->l) q.push(cur->l);
124             if (cur->r) q.push(cur->r);
125         }
126         level++;
127     }
128 }
129 int width() {
130     if (!root) return 0;
131     queue<Node*> q;
132     q.push(root);
133     int maxW = 0;
134     while (!q.empty()) {
135         int sz = q.size();
136         maxW = max(maxW, sz);
137         for (int i = 0; i < sz; i++) {
138             Node* cur = q.front(); q.pop();
139             if (cur->l) q.push(cur->l);
140             if (cur->r) q.push(cur->r);
141         }
142     }
143     return maxW;
144 }
145 };
146 int main() {
147     ifstream f("input.txt");
148     string line;
149     int caseNum = 1;
150     while (getline(f, line)) {
151         if (line.empty()) continue;
```

```
152     cout << "测试案例" << caseNum++ << endl;
153     cout << "输入表达式: " << line << endl;
154     Tree t;
155     t.buildFromString(line);
156     cout << "叶节点数目: " << t.countLeaves() << endl;
157     cout << "层次遍历: ";
158     t.levelOrder();
159     cout << "各层结点数:" << endl;
160     t.printLevelNodes();
161     cout << "交换左右子树后:" << endl;
162     t.swapAll();
163     t.printTree();
164     cout << "交换后的层次遍历: ";
165     t.levelOrder();
166     vector<char> swappedLeaves = t.getLeafNodes();
167     cout << " 交换后的叶子节点: ";
168     for (char c : swappedLeaves) {
169         cout << c << " ";
170     }
171     cout << endl;
172     cout << " 树的宽度: " << t.width() << endl;
173     cout << endl;
174 }
175 f.close();
176 system("pause");
177 return 0;
178 }
```

### 3.5 测试用例截图

